CARTRESCUE-AGENT — Diseño ampliado e implementación simulada

> Este informe parte del diseño inicial de Eli Bautista Welman Amaya y lo lleva hasta una simulación que se puede ejecutar y revisar. Código y documentación: github.com/Incomu1/CARTRESCUE. No se recibió retroalimentación docente; por eso, los ajustes se explican con base en los dos PDF entregados, el código local y la evidencia generada.

Registro de cambios

No hubo comentarios del docente para incorporar. Los cambios de esta tabla se hicieron al contrastar los dos PDF, revisar el código local y ordenar la evidencia sintética para que se pueda seguir con facilidad.

Sección	Qué decía	Qué dice ahora	Por qué cambió
Registro / selección	Se comparaba con una línea base construida por ausencia del segundo diseño.	Compara CARTRESCUE con el diseño real KnowKeeper y conserva la decisión explícita.	Se recibió Diseno_Agente.pdf; no corresponde inventar ni mantener un sustituto.
Selección Fase 1	No había matriz entre los dos diseños reales.	Matriz de seis criterios, pesos 25/20/20/15/10/10, fórmula, puntajes, totales y sensibilidad.	Hacer auditable la selección del equipo.
§8	Relación IBM resumida.	Mapea los siete componentes IBM a clases/métodos y distingue simulación de producción.	Evitar atribuir integraciones que el código no realiza.
§9.2–§9.4	Tecnologías, entradas, salidas y secuencia estaban	Tablas completas basadas en domain.py,	Alinear el informe con la implementación

Sección	Qué decía	Qué dice ahora	Por qué cambió
	resumidas.	acciones/efectos y diagrama Mermaid editable.	local real.
§9.5	Solo describía el protocolo de prueba.	Tabla completa de resultados, reformulación de SMART, media/DE y análisis de fallos.	Mostrar evidencia medible sin prometer resultados comerciales.
Guion/README	Guion breve y referencia a línea base construida.	Plan de grabación de dos integrantes, bitácora de IA y enlaces públicos pendientes.	Corregir el estado de los artefactos y no fabricar publicación.

Justificación de selección (Fase 1)

El equipo decidió implementar CARTRESCUE-AGENT. La matriz compara los dos PDF entregados: CARTRESCUE y Diseno_Agente.pdf (KnowKeeper). También toma en cuenta lo que ya se puede ejecutar en CARTRESCUE. Los puntajes son una **valoración del equipo**; no sustituyen una estimación independiente de costos ni prueban rendimiento en producción.

Criterio	Peso	CARTRESCUE-AGENT: puntaje y evidencia	KnowKeeper: puntaje y evidencia
Viabilidad de implementación	25%	**4/5.** Ya existe una simulación reproducible y el dominio se acota a eventos de carrito y políticas de contacto.	**2/5.** Requiere coordinar HR, mensajería, wiki, calendario y decisiones sensibles de privacidad laboral.
Datos y entorno	20%	**5/5.** El diseño define eventos de	**2/5.** Depende de datos de HR, chat, intranet y

Criterio	Peso	CARTRESCUE-AGENT: puntaje y evidencia	KnowKeeper: puntaje y evidencia
		carrito/checkout, inventario, preferencias y una versión sintética utilizable.	redes de comunicación cuyo acceso y gobernanza son complejos.
Medibilidad	20%	**4/5.** Tiene tasa de recuperación, cumplimiento y duplicados; la simulación aporta una corrida trazable.	**2/5.** Define metas SMART, pero sus resultados exigen horizontes de 30 días, seis meses y un año, además de etiquetas sensibles.
Complejidad de IA	15%	**4/5.** Combina aprendizaje, utilidad y restricciones en un entorno estocástico secuencial.	**5/5.** Integra riesgo, PLN, conocimiento tácito, priorización y coordinación multiagente.
Costo y recursos	10%	**5/5.** La versión académica usa Python estándar, datos sintéticos y no necesita integraciones.	**2/5.** Sus múltiples integraciones, controles de privacidad y revisión humana previsiblemente demandan más recursos.
Valor	10%	**4/5.** Ataca una	**5/5.** Puede

Criterio	Peso	CARTRESCUE-AGENT: puntaje y evidencia	KnowKeeper: puntaje y evidencia
		pérdida comercial concreta y medible: carritos recuperados.	preservar continuidad y conocimiento organizacional de alto impacto.

Fórmula: total = Σ((peso / 100) × puntaje), con puntajes de 1 a 5 y pesos que suman 100%.

Alternativa	Cálculo	Total ponderado
CARTRESCUE-AGENT	(25×4 + 20×5 + 20×4 + 15×4 + 10×5 + 10×4) / 100	**4.30** / **5**
KnowKeeper	(25×2 + 20×2 + 20×2 + 15×5 + 10×2 + 10×5) / 100	**2.75** / **5**

Sensibilidad del criterio más influyente: viabilidad

La viabilidad (25%) es el criterio de mayor peso. Se altera en ±10 puntos porcentuales y el cambio inverso se reparte por igual entre los otros cinco criterios.

Caso	Pesos [viabilidad, datos, medibilidad, complejidad, costo, valor]	CARTRESCUE	KnowKeeper	Decisión
Base	[25, 20, 20, 15, 10, 10]	4.30	2.75	CARTRESCUE
Viabilidad +10 pp	[35, 18, 18, 13, 8, 8]	4.26	2.63	CARTRESCUE
Viabilidad −10 pp	[15, 22, 22, 17, 12, 12]	**4.34**	2.87	CARTRESCUE

Verificación aritmética: con los puntajes y pesos anteriores, el último total de CARTRESCUE es 0.15×4 + 0.22×5 + 0.22×4 + 0.17×4 + 0.12×5 + 0.12×4 = 4.34. Por ello se

registra 4.34, no 4.54: 4.54 sería incompatible con la matriz solicitada. La decisión se mantiene en todos los casos. El intercambio aceptado es renunciar, en esta primera implementación, al valor potencial de KnowKeeper para la continuidad y preservación del conocimiento tácito organizacional.

1. Introducción y planteamiento del problema

Cuando una persona deja productos en el carrito, la tienda pierde una oportunidad de venta, pero no siempre conviene insistir. CARTRESCUE-AGENT usa las señales disponibles del carrito para decidir si vale la pena intervenir y con qué acción. Lo que la persona piensa o hace fuera de la tienda no es visible para el sistema; esa es una de las limitaciones del problema.

La propuesta no busca enviar más mensajes por defecto. Busca elegir una acción solo cuando hay consentimiento, un canal disponible, un contexto válido y una utilidad esperada positiva.

2. Tipo de agente inteligente

Se clasifica principalmente como un **agente de aprendizaje** porque registra si cada intervención termina o no en recuperación y ajusta su estimación por tipo de acción. También trabaja con una meta y una función de utilidad: no aplica una única regla, sino que compara alternativas según el margen, el costo y la probabilidad estimada.

Para esta entrega se eligió un mecanismo sencillo y revisable, no un modelo opaco. Así se puede seguir el camino desde una señal del carrito hasta la decisión tomada.

3. Tipos de ambiente

Propiedad	Clasificación	Consecuencia de diseño
Observabilidad	Parcial	Las motivaciones no son visibles; se usan solo señales digitales permitidas.
Determinismo	Estocástico	Una acción puede o no recuperar; se fija semilla para repetir la simulación.
Episodicidad	Secuencial	El historial de contactos

Propiedad	Clasificación	Consecuencia de diseño
		y resultados modifica la siguiente decisión.
Dinamismo	Dinámico	Inventario, checkout y disponibilidad se validan antes de actuar.
Discretización	Mixto, principalmente continuo	Tiempo, probabilidad y valor son numéricos; la acción es discreta.
Número de agentes	Único	Servicios de correo, push e inventario son entorno/herramientas, no cooperadores.

4. Descripción del entorno

El entorno representa una tienda en línea con carrito, checkout, inventario, historial agregado y canales autorizados. Si faltan datos, no hay consentimiento, se supera la frecuencia permitida, el checkout sigue activo, no hay inventario o ya existe una intervención, el agente se detiene y deja el motivo en el registro.

Alcance de esta entrega: los valores de carrito y margen son sintéticos. No hay conexión con una tienda, ni se contacta a personas reales.

5. Entradas y sensores

Entrada/sensor	Uso permitido
API/eventos de carrito y checkout	valor, inactividad, estado del checkout
Historial agregado	compras previas y respuesta histórica
Inventario/reglas	elegibilidad de oferta y disponibilidad
Preferencias de contacto	consentimiento, canal, límite y enfriamiento

No se usan nombre, correo, teléfono, dirección ni contenido personal. En producción serían necesarios minimización de datos, retención definida y controles de acceso.

6. Salidas y actuadores

Acciones posibles: email, push, incentive, recommendation o none. Los actuadores se representan como eventos de simulación, no como integraciones reales. Toda acción pasa antes por el portón de política; none es una decisión válida.

7. Medidas de desempeño y decisión

La meta SMART se mantiene: en tres semanas, alcanzar >=**15%** de recuperación atribuible de oportunidades intervenidas, sin incumplir políticas ni duplicar intervenciones.

Métricas predeclaradas

Métrica	Fórmula	Umbral
Recuperación	recuperados / intervenidos	>= 0.15
Cumplimiento de política	bloqueos con motivo correcto / bloqueos	1.00
Duplicados	acciones ejecutadas duplicadas	0
Ingreso recuperado	suma de valor del carrito recuperado	informativo
Costo de incentivo	suma de costos de acciones	informativo

Función de utilidad

U(a) = P(recuperación | a, señales) × margen esperado − costo(a) − penalización de frecuencia.

Primero se filtran restricciones duras: consentimiento, máximo 2 contactos/7 días, 24 h de enfriamiento, canal, inventario, checkout, datos completos y no duplicación. Luego se comparan utilidades. La posterior por acción inicia en Beta(3,7) y se actualiza con cada resultado; eso implementa el ciclo percepción–acción–retroalimentación de forma verificable.

Matriz de decisión y sensibilidad

Condición	Acción esperada	Razón
Sin consentimiento	none	Restricción legal/ética

Condición	Acción esperada	Razón
		dura, no compensable por valor.
Límite de frecuencia o 23 h desde contacto	none	Reduce fatiga y cumple política.
Alto margen, elegible	Puede ganar incentive	Mayor lift puede compensar costo.
Bajo margen	Excluye incentive	El costo puede destruir utilidad.
Solo push disponible	push o none	Respeta canal autorizado.

Sensibilidad: al subir el costo del incentivo o bajar el margen, incentive pierde utilidad; al aumentar el historial de respuesta o la inactividad, sube la probabilidad estimada. Ningún cambio de sensibilidad puede eludir un bloqueo de política.

8. Componentes de la arquitectura de IA agéntica (marco IBM)

Componente IBM	Código/artefacto actual	Estado y límite de producción
Percepción y manejo de entradas	CartContext en cartrescue/domain.py; default_scenarios() fabrica las señales.	**Simulado.** No hay API de tienda, navegador, cliente ni datos reales.
Planificación y descomposición	CartRescueAgent.decide() ordena: elegibilidad → candidatos → utilidad → decisión.	**Implementado en simulación.** No hay orquestador de tareas de producción.
Memoria	posteriors Beta-Bernoulli del agente y Experiment.records/events.	**En memoria durante la ejecución.** No hay base de datos, retención ni memoria entre procesos.
Razonamiento y toma de decisiones	eligibility(), probability() y utilidad en agent.py.	**Implementado en simulación.** Es una heurística auditable, no un modelo validado.

Componente IBM	Código/artefacto actual	Estado y límite de producción
Llamada de herramientas y acción	Experiment escribe action_executed con synthetic_channel=True.	**No implementado en producción.** No invoca correo, push, inventario ni APIs; así evita contacto real.
Comunicación	Las acciones email/push existen como etiquetas y logs.	**No implementado en producción.** No hay destinatario, plantilla, proveedor ni envío.
Aprendizaje y adaptación	learn(action, recovered) actualiza la posterior después de un resultado.	**Implementado solo en simulación.** No hay entrenamiento offline, monitoreo de deriva ni aprobación humana.

La designación “implementado” significa que existe en el simulador local; no implica un servicio desplegado ni una integración productiva.

9. Implementación de CARTRESCUE-AGENT

9.1 Comparación y selección: diseños reales

La Fase 1 compara los dos diseños que entregó el equipo: CARTRESCUE-AGENT y KnowKeeper. La matriz, la fórmula y la sensibilidad aparecen antes de la sección 1 para que la decisión se entienda antes de entrar al código.

Alcance: KnowKeeper se usa como alternativa para la comparación. No se escribió código, simulación, integración ni despliegue de KnowKeeper en este repositorio. De la sección 9.2 a la 9.7 se documenta únicamente CARTRESCUE-AGENT.

9.2 Arquitectura técnica (solo CARTRESCUE)

```
[Escenarios sintéticos] -> [CartContext / percepción] -> [portón de política]
                                                          | bloqueado -> [log timestamped]
                                                          v
                                                [posterior + utilidad] -> [acción simulada]
                                                          -> [entorno estocástico]
                                                          -> [resultado] -> [actualización Beta]
```

```
CLI --seed/--runs -> Experiment -> CartRescueAgent -> evidence/
                    |              |              |- events.ndjson
                    |              |              |- scenario_results.csv
                    |              |              `- summary.csv/json
                    |              `-> 12 escenarios x >=5 repeticiones
```

El diagrama de secuencia editable es diagrama_secuencia_cartrescue.mmd. Describe solo llamadas reales del simulador; el actor externo queda marcado como **no invocado**.

```
sequenceDiagram
    actor U as Usuario / contexto sintetico
    participant CLI as CLI local
    participant X as Experiment
    participant A as CartRescueAgent
    participant E as Entorno estocastico
    participant S as Servicio externo
    U->>CLI: ejecuta --all --runs 5 --seed 20261019
    CLI->>X: run(default_scenarios(), 5)
    loop 5 repeticiones x 12 escenarios
        X->>A: decide(CartContext)
        A->>A: eligibility() y puntuar utilidad
        alt politica bloquea
            A-->>X: Decision(none, motivo)
            X->>X: log cart_detected + decision
        else elegible
            A-->>X: Decision(accion, utilidad)
            X->>E: _outcome(escenario, decision)
            E-->>X: Outcome(recuperado/no)
            X->>X: log action_executed + outcome_observed
            X->>A: learn(accion, recuperado)
        end
    end
    Note over X,S: No hay llamada real a S; synthetic_channel=True solo registra la simulacion.
    X-->>CLI: CSV, JSON y NDJSON
```

9.3 Tecnologías, entradas y salidas (solo CARTRESCUE)

Tecnologías

Tecnología	Versión local probada	Rol	Elección y tradeoff frente a alternativa
Python	**3.14.4** (python3 --version)	Ejecuta CLI, agente, simulación, exportación de evidencia y pruebas.	Biblioteca estándar: máxima reproducibilidad y cero instalación. Frente a un framework ML, se sacrifica modelado avanzado por auditabilidad.
dataclasses, typing	stdlib de Python 3.14.4	Contratos inmutables CartContext, Decision, Outcome.	Frente a Pydantic/ORM, no valida rangos en runtime ni persiste datos; evita dependencia externa.
random, datetime	stdlib de Python 3.14.4	Entorno estocástico con semilla y timestamps UTC deterministas.	Frente a servicios/eventos reales, no representa tráfico ni reloj real.
csv, json	stdlib de Python 3.14.4	Evidencia legible: CSV, JSON y NDJSON.	Frente a una BD/warehouse, no permite consultas operativas ni retención.
unittest	stdlib de Python 3.14.4	Pruebas de política, semilla y evidencia.	Frente a frameworks externos, hay menos ergonomía/plug-

Tecnología	Versión local probada	Rol	Elección y tradeoff frente a alternativa
			ins pero ninguna dependencia.
Pandoc + LibreOffice	Herramientas locales de documentación	Exportan el informe Markdown a DOCX/PDF.	No forman parte del runtime del agente ni de requirements.txt.

Dependencias de runtime: ninguna fuera de la biblioteca estándar. requirements.txt lo declara explícitamente.

Entradas: contrato CartContext

Todos los campos proceden de default_scenarios() y son sintéticos; los ejemplos corresponden al catálogo local, no a personas ni carritos reales.

Campo	Tipo y rango/valores	Fuente actual	Ejemplo sintético
session_id	str; formato rNN-II-escenario durante una corrida	Generado en Experiment.run()	r01-01-normal_email
scenario	str; uno de 12 nombres del catálogo	default_scenarios()	normal_email
category	str; normal, boundary, failure	default_scenarios()	normal
cart_value	float; > 0 en catálogo	Parámetro value sintético	65.0
expected_margin	float; > 0 en catálogo	Parámetro margin sintético	25.0
inactivity_minutes	int; >= 0	Parámetro inactivity sintético	90
prior_purchases	int; >= 0	Parámetro purchases sintético	3
campaign_response_rate	float; [0, 1]	Parámetro	0.35

Campo	Tipo y rango/valores	Fuente actual	Ejemplo sintético
		response sintético	
has_consent	bool	Parámetro consent sintético	False en sin_consentimiento
contacts_last_7_days	int; >= 0	Parámetro contacts sintético	2 en limite_contactos
hours_since_last_contact	Optional[int]; None o >= 0	Parámetro last sintético	23 en enfriamiento_23h
email_available	bool	Parámetro email sintético	False en normal_push
push_available	bool	Parámetro push sintético	False en canales_no_disponibles
inventory_available	bool	Parámetro inventory sintético	False en inventario_no_disponible
checkout_active	bool	Parámetro checkout sintético	True en checkout_activo
data_complete	bool	Parámetro complete sintético	False en datos_incompletos
duplicate_intervention	bool; por defecto False	Parámetro duplicate sintético	True en intervencion_duplicada

El código declara los tipos, pero no ejecuta validación de rango; los límites de la tabla documentan el contrato esperado para el catálogo actual.

Salidas, acciones y efectos

Salida/acción	Valores o efecto de código	Efecto real de la corrida de referencia	Límite
Decision.action	none, email, push, incentive, recommendation	20 acciones: 18 recommendation, 1 push, 1 incentive, 0 email.	Etiquetas de simulación; no mandan mensajes.
Decision.reason	Bloqueos de	40 bloqueos con	No equivale a un

Salida/acción	Valores o efecto de código	Efecto real de la corrida de referencia	Límite
	política o maxima_utilidad_con_politica	motivo registrado.	rechazo de proveedor externo.
Decision.estimated_probability / expected_utility	float; se escriben redondeados a 4 decimales en NDJSON	Trazan la base de decisión por sesión.	No son probabilidades calibradas con datos reales.
action_executed	Evento solo si action != none, con synthetic_channel=True	20 eventos; costo aplicado según acción.	No hay actuador/API real.
Outcome	recovered, ingreso atribuible y costo	7 recuperados; $875.00 sintéticos; costo $8.91 sintético.	La atribución proviene de _outcome(), no de una compra real.
Archivos de evidencia	events.ndjson, scenario_results.csv, summary.csv/json	160 eventos UTC; 60 filas de resultados.	Son artefactos locales, no tablero operativo.

9.4 Escenarios, resultados y protocolo de prueba (solo CARTRESCUE)

La corrida de referencia usa 12 escenarios: 4 normales, 3 de frontera y 5 de fallo. Al repetirlos cinco veces se obtienen **60 oportunidades sintéticas**. Con la semilla 20261019 se registran 160 eventos: 60 cart_detected, 60 decision, 20 action_executed y 20 outcome_observed. Los timestamps de events.ndjson están en UTC (+00:00). En los cinco escenarios de fallo el agente registra el motivo y no ejecuta ninguna acción.

Comandos reproducibles:

```
python3 -m unittest discover -s tests -v
python3 -m cartrescue --all --runs 5 --seed 20261019 --output evidence/reproducible_run
```

Reformulación de las medidas originales

La meta original habla de tres semanas y de resultados comerciales. Esta evidencia **no equivale a tres semanas reales**: son cinco repeticiones dentro de un simulador. Por eso, cuando la tabla indica “cumple”, significa únicamente que el umbral se alcanzó bajo estas condiciones. Tampoco existe un proveedor de

mensajería contra el cual medir rechazos; en su lugar se muestran las reglas de política que sí se pueden revisar en los logs.

Clase	Medida adoptada	Fórmula	Umbral predeclarado	Cobertura	Resultado numérico	Cumplimiento	Evidencia
SMART original (proxy)	Tasa de recuperación atribuible	recuperados / intervenidos	>= 0.15	12 × 5; 20 intervenidos	7 / 20 = **0.35**	Sí, solo en simulación	evidence/ reproducible_run/ summary.json
SMART original (proxy)	Conversión posterior a intervención	recuperados / intervenidos	Sin segundo umbral independiente	12 × 5; 20 intervenidos	**0.35**	Informativo; misma definición operacional anterior	summary.json, scenario_results.csv
Complementaria original	Valor de ventas recuperadas	Σ attributable_revenue recuperado	Informativo	12 × 5	**875.00** sintético	N/A	scenario_results.csv
Complementaria original	Costo de incentivos/acciones	Σ incentive_cost en intervenidos	Informativo	12 × 5	**8.91** sintético	N/A	summary.json
Complementaria original	Comunicaciones rechazadas/no	rechazo externo / comuni	No medible sin proveedor	12 × 5	**N/A**	No evaluable; no se afirma	Código: synthetic_channel=True

Clase	Medida adoptada	Fórmula	Umbral predeclarado	Cobertura	Resultado numérico	Cumplimiento	Evidencia
	autorizadas	caciones enviadas				0 rechazos	
Salvaguarda derivada	Cumplimiento de política en bloqueos	bloqueos con motivo válido / bloqueos	= 1.00	40 bloqueos	40 / 40 = **1.00**	Sí	summary.json, events.ndjson
Salvaguarda derivada	Intervenciones duplicadas	acciones para escenario marcado duplicado	= 0	5 casos intervencion_duplicada	**0**	Sí	scenario_results.csv
Informativa	Oportunidades / intervenciones	conteos de registros / acciones no none	Informativo	12 × 5	**60** / **20**	N/A	summary.json
Informativa	Media y DE de recuperación por repetición	media y DE muestral de [recuperados/intervenidos]	Informativo	5 tasas: 0.25, 0.50, 0.00, 0.50, 0.50	**0.350** / **0.224**	N/A	Derivado de scenario_results.csv
Inform	Media	media	Inform	5	**175.00** /	N/A	Deriva

Clase	Medida adoptada	Fórmula	Umbral predeclarado	Cobertura	Resultado numérico	Cumplimiento	Evidencia
ativa	y DE de ingreso recuperado por repetición	y DE muestral de ingresos por repetición	ativo	repeticiones	**137.07**		do de scenario_results.csv
Informativa	Media y DE de costo por repetición	media y DE muestral de costos por repetición	Informativo	5 repeticiones	**1.782** / **3.537**	N/A	Derivado de scenario_results.csv

Método de DE: desviación estándar muestral (n−1) de las cinco repeticiones, calculada sobre las filas existentes de scenario_results.csv; no es intervalo de confianza ni estimación poblacional.

9.5 Análisis de fallos (solo CARTRESCUE)

Observación real	Causa en el simulador	Cambio propuesto antes de producción
40 de 60 oportunidades quedaron bloqueadas.	Ocho restricciones de seguridad/estado dominan la utilidad: consentimiento, frecuencia, enfriamiento, canal, inventario, checkout, datos o duplicación.	Mantener bloqueos duros; validar políticas y consentimiento contra una fuente autorizada antes de cualquier envío.
13 de 20 intervenciones no recuperaron el carrito.	_outcome() usa una probabilidad estocástica por escenario; la acción no garantiza compra.	Evaluar con A/B test, atribución acordada y tamaños muestrales suficientes; no

Observación real	Causa en el simulador	Cambio propuesto antes de producción
		interpretar 5 repeticiones como eficacia comercial.
La recuperación por repetición tiene DE 0.224.	Muestra pequeña y dispersión de resultados [0.25, 0.50, 0.00, 0.50, 0.50].	Aumentar repeticiones sintéticas para pruebas y, en un estudio real, informar intervalos de confianza y segmentación.
Hubo costo alto en la primera repetición (incentivo) y costo total 8.91.	El agente actualiza posterior por acción; las decisiones cambian a lo largo de la secuencia.	Añadir presupuesto, límites de descuento, exploración explícita y auditoría de equidad antes de usar incentivos reales.
No existe tasa real de rechazo de comunicaciones.	No hay proveedor, destinatario ni envío: synthetic_channel=True.	Integrar sandbox autorizado, callbacks de entrega/rechazo y revisión de privacidad; nunca inferir esta métrica de los logs actuales.

9.6 Limitaciones y trabajo futuro (solo CARTRESCUE)

- Las probabilidades/lifts son supuestos sintéticos, no estimaciones con clientes reales.
- La actualización por acción no controla causalidad, sesgo de selección ni estacionalidad.
- No hay envío de correo/push, base de datos, autenticación, dashboard ni despliegue.
- Falta validar reglas reales comerciales, privacidad, consentimiento y retención.
- Se debe añadir atribución experimental, límites por jurisdicción, revisión humana y alertas de degradación antes de producción.

9.7 Enlaces y entregables pendientes (solo CARTRESCUE)

- Evidencia local: ../evidence/reproducible_run/summary.json.
- Repositorio público: https://github.com/Incomu1/CARTRESCUE.
- Guion de video: GUIA_GRABACION.md. **El integrante responsable aún debe grabar y publicar el video.**
- Despliegue: **pendiente; no se ha implementado un despliegue público.**

10. Conclusiones

CARTRESCUE-AGENT funciona como una primera propuesta porque une percepción, reglas, utilidad, acción y aprendizaje en un problema concreto. La diferencia frente a una regla fija no es enviar más mensajes: es respetar los límites de contacto y decidir dentro de ellos.

La simulación permite revisar el comportamiento con supuestos claros, pero no demuestra efectividad en producción. Antes de usarlo fuera del curso haría falta un experimento controlado, datos minimizados, consentimiento verificable y criterios de atribución acordados.

Referencias

- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th Global ed.), cap. 2.
- IBM. ¿Cuáles son los componentes de los agentes de IA?.
- IBM. Agentes de IA.
- Catálogo del curso: *Intelligent Agents / Use Cases*, caso CARTRESCUE-AGENT.